

Theoretische Informatik II

Markus Hecher

Universität Potsdam, Theoretische Informatik, SoSe 2026

Datum: 2026-07-06

Name:

Matrikelnummer:

Aufgabe 1 (Wahr-Falsch Fragen)

Kreuzen Sie bei folgenden Aussagen jeweils an, ob diese zutreffen ("Ja") oder nicht zutreffen ("Nein").

(9 Punkte)

Ja Nein

- ☒ ☒ In der Komplexitätstheorie befasst man sich mit Laufzeit- bzw. Speicherplatzverbrauch von berechenbaren Problemen.
- ☐ ☒ Aus Sicht einer Turingmaschine fragt NP nach der Nichtexistenz eines akzeptierenden Pfades.
- ☒ ☐ Kompakte Zertifikate und Überprüfbarkeit dieser Zertifikate in Polynomialzeit sind die Schlüsseleigenschaften, die NP charakterisieren.
- ☐ ☒ Dadurch, dass man ein beliebiges NP-Problem in konstanter Zeit invertieren kann, können wir direkt das inverse Problem in NP berechnen.
- ☒ ☐ Die Entscheidung, ob eine Instanz eines NP-harten Problems positiv ist, kann man in konstanter Zeit invertieren.
- ☐ ☒ Probleme, die härter sind als NP, können nicht NP-hart sein.
- ☒ ☐ Um zu zeigen, dass ein Problem (wahrscheinlich) schwierig zu berechnen ist, benötigen wir keine NP-Vollständigkeit, sondern NP-Härte ist ausreichend.
- ☒ ☐ Um $P=NP$ zu zeigen, würde es genügen, ein NP-hartes Problem in Polynomialzeit ohne Verwendung von Nichtdeterminismus zu lösen.
- ☐ ☒ Jedes NP-harte Problem kann per Definition in Polynomialzeit in jedes andere NP-harte Problem reduziert werden.
- ☐ ☒ Wenn eine Boolesche Formel gültig ist, kann die negierte Formel auch erfüllbar sein.
- ☒ ☐ Sowohl NP als auch coNP sind in ExpTime enthalten, weil man einfach zeigen kann, dass man jedes Problem in einer dieser Klassen in exponentieller Zeit lösen kann.
- ☒ ☐ Die Klasse coNP besteht aus solchen Problemen, deren inverses Problem (positive und negative Instanzen vertauscht) in NP ist.
- ☐ ☒ Sollte $NP=coNP$ gelten, muss auch $P=NP$ gelten.
- ☒ ☐ Sollte $P=NP$ gelten, muss auch $NP=coNP$ gelten.
- ☐ ☒ Härte für eine Klasse liefert eine erste obere Schranke.
- ☒ ☐ Wenn wir ein Problem $\mathcal{P}$ auf ein bekanntes Problem $\mathcal{P}'$ reduzieren (via $\leq_p$) und wir wissen, dass $\mathcal{P}' \in NP$, dann ist auch $\mathcal{P}$ in NP.
- ☐ ☒ Härte eines Problems $\mathcal{P}$ für eine Komplexitätsklasse C können wir direkt durch Reduktion auf ein bekanntes C-hartes Problem $\mathcal{P}'$ zeigen.
- ☐ ☒ Wenn wir zeigen können, dass ein Problem $\mathcal{P}$ Mitglied einer Klasse C ist, dann sind auch jene Probleme, deren Instanzen eine Übermenge der Instanzen von $\mathcal{P}$ sind, in C.

Aufgabe 2 (Problemreduktion)

Sei 3-SAT das Problem, das für eine gegebene Boolesche Formel in 3-CNF fragt, ob die Formel erfüllbar ist. Sie kennen auch bereits das Vertex Cover Problem, das für einen gegebenen ungerichteten Graphen fragt, ob es eine Knotenüberdeckung (ein Vertex Cover) einer bestimmten Größe k gibt (d.h., für jede Kante ist zmd. ein Endpunkt in der Lösungsmenge).

- a) Ist das dem Vertex Cover unterliegende Optimierungsproblem eigentlich ein Minimierungs- oder Maximierungsproblem? D.h., sind kleine oder große Werte für k potenziell schwieriger zu lösen?

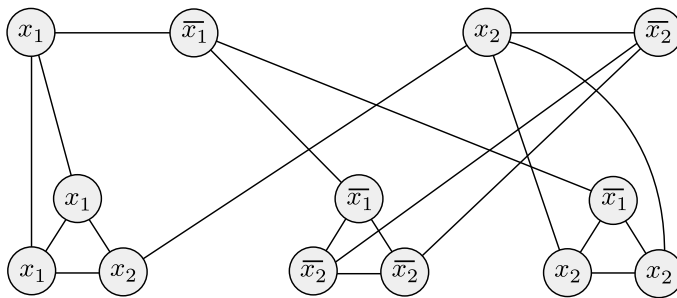
(2 Punkte)

b) Angenommen wir hätten eine Reduktion von 3-SAT auf Vertex Cover. Was zeigt diese für Vertex Cover? (2 Punkte)
NP-Härte

c) Was würde zusätzlich zur Reduktion noch fehlen, um NP-Vollständigkeit nachweisen zu können? Argumentieren Sie kompakt, warum dieser fehlende Teil für Vertex Cover gilt. (3 Punkte)

NP-Mitgliedschaft. Es genügt eine kompakte Lösung $I \subseteq V$ (kann maximal Instanzgröße $|V|$ haben) zu raten und in Polynomzeit (naiv quadratisch) für jede Kante überprüfen, ob ein Endpunkt in I liegt.

d) Beweisen Sie beide Richtungen der Korrektheit der folgenden Reduktion 3-SAT auf Vertex Cover: Für jede der n vielen Variablen legen wir neue Knoten x und $\bar{x}$ (verbunden durch Kante) an. Für jede der m vielen Klauseln gibt es ein neues Dreieck, d.h. jedes Literalvorkommen bekommt einen neuen Knoten. Variablenknoten werden direkt mit den jeweiligen Vorkommnissen verbunden. Wir setzen $k = 2m + n$. Die Reduktion ist für $(x_1 \vee x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_2)$ skizziert. (6



Punkte)

Wir reduzieren eine beliebige 3-SAT Formel φ auf eine Vertex-Cover Instanz $R(\varphi) = (G, k)$, wobei R bereits oben definiert ist. Wir nennen die Knoten, die für eine Variable erzeugt werden Variablenknoten und die anderen Klauselnknoten. Es verbleibt zu zeigen, dass φ erfüllbar $\iff (G, k)$ hat ein Vertex Cover.

“ $\implies$ ”

1. Vorbedingung. Sei I eine erfüllbare Belegung von φ . Dann wissen wir, dass jede Klausel c in φ unter I erfüllt ist, d.h., mindestens eines ihrer Literale ℓ wertet durch I zu $true$ aus.

2. Konstruktion. Wir konstruieren ein Vertex Cover C : Für jede Variable x von φ enthält C Variablenknoten x wenn $I(x) = true$, andernfalls Variablenknoten $\bar{x}$ (weil $I(x) = false$). C enthält genau zwei Klauselnknoten ℓ_1, ℓ_2 jeder Klausel $c = \ell_1 \vee \ell_2 \vee \ell_3$, solange ℓ_3 unter I zu $true$ auswertet.

3. Konstruktion beweisen. Es verbleibt zu zeigen, dass C ein Vertex Cover von G ist. Widerspruch: Angenommen, es gäbe eine Kante $e = \{v, \ell\}$ in G , die nicht durch C abgedeckt wäre. Per Konstruktion kann e nicht zwischen Variablenknoten sein (I ist erfüllbare Belegung; jede Variable ist $true$ oder $false$). Kante e kann per Konstruktion keine Dreieckskante sein. Kante e kann daher nur zwischen Klauselnknoten v und Variablenknoten ℓ sein. Dafür muss $\{v, \ell\} \cap C = \emptyset$, da e sonst abgedeckt ist. Per Konstruktion von C wertet damit die zum Dreieck zugehörige Klausel c unter I zu $false$ aus. Das widerspricht, dass I erfüllbare Belegung ist.

“ $\impliedby$ ”

1. Vorbedingung. Sei C ein Vertex Cover von G , sodass $|C| \leq k$. Beobachtung 1: In jeder Kante muss eine der beiden Knoten in C sein; in jedem Dreieck müssen zwei der Knoten in C sein.

2. Konstruktion. Wir konstruieren eine erfüllbare Belegung I : Eine Variable x in φ wird auf $true$ gesetzt genau dann wenn der für x konstruierte Variablenknoten x in C vorkommt. Durch Beobachtung 1 und $k = n + 2m$, kann C für eine Variable x nicht zugleich Variablenknoten $x, \bar{x}$ enthalten. Daher ist I wohl definiert (jede Variable hat Wert).

3. Konstruktion beweisen. Es genügt zu zeigen, dass φ unter I zu $true$ auswertet. Widerspruch: Angenommen φ werte unter I zu $false$ aus. Dann gibt es eine Klausel c , sodass alle drei Literale unter I zu $false$ auswerten. Durch Beobachtung 1 kann C allerdings nur 2 Klauselnknoten pro Klausel enthalten. Es muss daher im Dreieck für c einen Knoten v geben, der nicht in C ist, sodass der benachbarte Variablenknoten mit Label ℓ auch nicht in C ist. (Andernfalls würde Literal ℓ in c unter I per Konstruktion von I zu $true$ auswerten). Das lässt aber die Kante $\{v, \ell\}$ nicht abgedeckt, weil $\{v, \ell\} \cap C = \emptyset$, was allerdings widerspricht, C wäre Vertex Cover von G .